Deep Think with Confidence (DeepConf)

Paper Summary & Study Report

Course: Deep Learning | CMPE 258 | San Jose State University
Paper: arXiv:2508.15260 | Fu et al., Meta AI, August 2025

Title	Deep Think with Confidence (DeepConf)
Authors	Yichao Fu (UCSD), Xuewei Wang, Yuandong Tian, Jiawei Zhao (Meta AI)
Institution	Meta AI Research (FAIR) & UC San Diego
Published	August 21, 2025 — arXiv:2508.15260 [cs.LG]
Topic	Test-Time Scaling / Efficient LLM Reasoning / Confidence Estimation
Project	jiaweizzhao.github.io/deepconf
Submitted By	Prachi Gupta
SJSU ID	019106594

1. Introduction and Motivation

I decided to search for some new information concerning scaling during testing time, and I came across this specific paper, whose central idea instantly attracted my interest. The point is Large Language Models can reason, but it takes much effort to do it accurately and precisely. Self-consistency with majority voting is a common technique utilized by researchers, according to which one provides the same prompt for the model several times, receives several reasoning paths, and then selects the response that occurs most frequently. While the technique might seem sensible and efficient in practice, it is expensive.

For instance, one may refer to the paper discussed above, where in order to ensure that Qwen3-8B reaches 82% performance on AIME 2025, one needs to expend an extra 511 reasoning traces (or more than 100 million tokens) for each sample by using majority voting. However, what makes the issue more problematic is that there does not seem to exist any direct relationship between number of traces and their efficiency, since at a certain point, their quantity no longer affects accuracy, and the latter saturates.

After seeing the explanation for why that was the case, everything became crystal clear to me. With the standard approach of majority voting, all the traces have the same voting power, regardless of how they were generated. The trace where the model logically thought about each step without any doubts gets counted the same way as the trace where the model kept changing its mind, second-guessing, backing out, and finally getting to the answer by saying "wait a minute, maybe I should rethink this." If there are too many such traces supporting the wrong answer, it could affect the voting results.

> **The Key Concept**
> Not all reasoning paths have equal value. The probability estimates assigned by the model to individual tokens during reasoning contain a hint of confidence that can be used to determine which reasoning paths should be trusted enough to cast their votes.

2. Background — What is Test-Time Scaling?

Before going on to describe DeepConf, I believe it is important to place it in context first. DeepConf falls under the category of test-time scaling, which posits that rather than spending all the compute budget for training the model, we should spend the budget at test-time to extract as much accuracy as possible from an existing model. Here are two ways to achieve test-time scaling:

- **Depth scaling:** Increase the depth of the reasoning chain. That means more steps, more reflection for each trace. All the large o1-style models like DeepSeek-R1, Kimi K1.5, Qwen3, and GPT-o family use this.
- **Breadth scaling (parallel thinking):** Generate multiple independent reasoning traces and merge them. Self-consistency and best-of-n sampling fall under this category.

DeepConf is definitely a breadth scaling technique. It does not modify the model architecture nor requires any kind of fine-tuning, external reward model, or verifier.

The most relevant previous work is Self-Certainty by Kang et al. (2025). This paper, too, employs token probabilities to measure trace reliability. However, there is a fundamental difference between

DeepConf's approach and the method used in Self-Certainty. According to DeepConf, the global average fails to capture a lot of important information. Instead, what matters is what occurs locally during reasoning, particularly in case of an error in between.

3. How Confidence Works as a Quality Signal

For each token generated, the model emits a probability distribution over the full vocabulary; this is inherent to autoregressive decoding. Usually, we simply pick the highest-probability token and continue from there. The probability distribution is interesting on its own terms, however; if the model is highly confident, nearly all probability will cluster on a single token, whereas if the model is less sure, the probability mass will be distributed more widely. DeepConf exploits this.

Token Entropy

Entropy Hi = -(Σ Pi(j) * log Pi(j)) where the sum ranges over all vocabulary tokens j. A low entropy means high confidence, whereas a high entropy means low confidence — this is actually Shannon entropy applied directly to the output distribution.

Token Confidence

Confidence Ci = -(1/k)(Σ log Pi(j)) for the top k tokens (or simply: negative average log-prob). High confidence here would mean a model has strong convictions about which token comes next, while low confidence indicates it's less sure.

The important empirical discovery which underlies the whole method can be seen clearly illustrated in Figure 2 of the paper: when comparing distributions of token confidences for correct vs. incorrect reasoning traces, the latter have shifted to the right significantly.

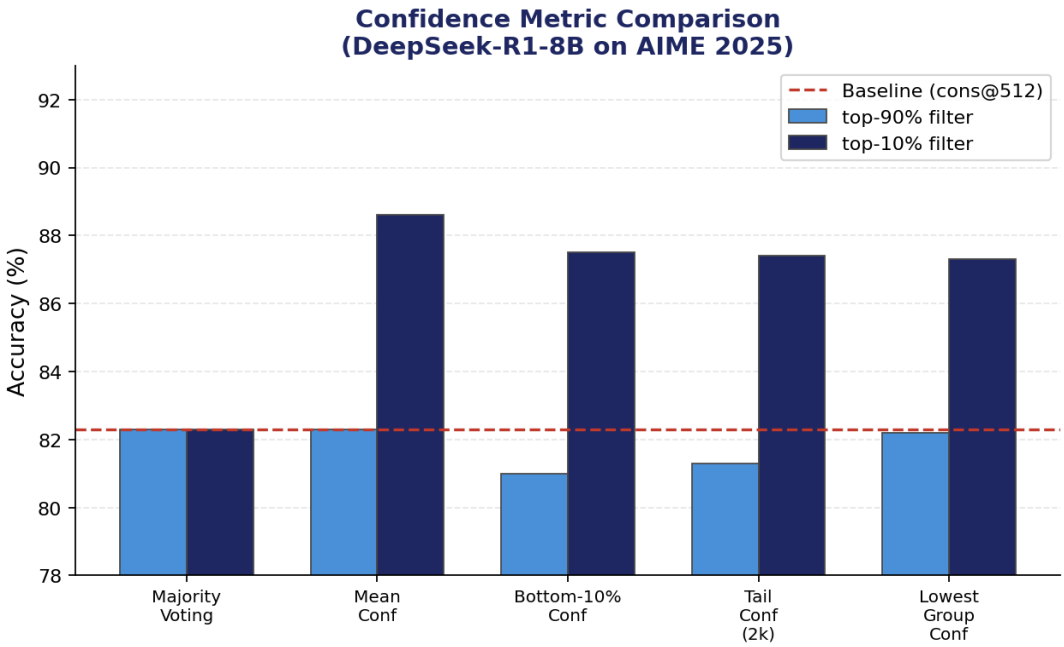

Figure 1: Different confidence metrics compared on DeepSeek-R1-8B, AIME 2025 — local metrics consistently beat the global mean baseline

4. DeepConf's Confidence Measurements

It's also worth noting how much this paper goes into details on how confidence scores should be averaged throughout the entire trace. Specifically, the four different methods of confidence score are considered:

4.1 Average Trace Confidence

C_avg = (1/N)*sum of C_i for all N tokens. While this is a good baseline to use when comparing other values, because it is very similar to what the Self-Certainty framework does, it is still rather easy to manipulate for some models; for example, a model can begin its task confidently but then not perform well towards the end. In other words, it ignores the particularities of some bad performance.

4.2 Group Confidence (Sliding Window)

Instead of finding an average confidence score of all the tokens, the confidence is measured using a sliding window method with a span of n=1024 or 2048 tokens. Consequently, each token i is divided into a group G_i containing n tokens before it, and then the average confidence score of those n tokens serves as the confidence measure.

4.3 Confidence for Bottom 10% Groups

Instead of asking 'what's the mean confidence over the entire trace?', we can ask 'what's the confidence when things are going worst?'. We gather all the group confidences over the trace and select the bottom 10% for averaging. The logic behind this metric is the following: a trace that has an unusually long period of low confidence — where the model thinks 'no, wait, I should re-think this'— must be a bad trace regardless of how well it recovered later on.

4.4 Tail Confidence

Take the confidence of **the last K tokens** in the trace, averaged (K = 2048). The idea is based on the natural location of mathematics reasoning, i.e., in the process of assembling an answer, proving a theorem, etc. Thus, even though the model initially did great reasoning work but messed up in the end, this should be considered a failed trace. On the other hand, tail confidence will serve as evidence that the model knew what it was doing.

4.5 Least Group Confidence

C_least = min over all group windows G_j. In other words, the worst group confidence window in the whole trace. That's about as extreme as finding 'the worst moment' gets. This measure would be particularly valuable when working in the online mode, since you can calculate it continually while generating your trace and shut down whenever the current window's confidence falls beneath a certain level.

> **Why Local Metrics > Global Metrics**
> Bottom-10% confidence and tail confidence distinguish between correct and incorrect traces better than mean confidence. Even one low-confidence region in the middle of the trace is a big red flag.

5. How DeepConf Actually Works

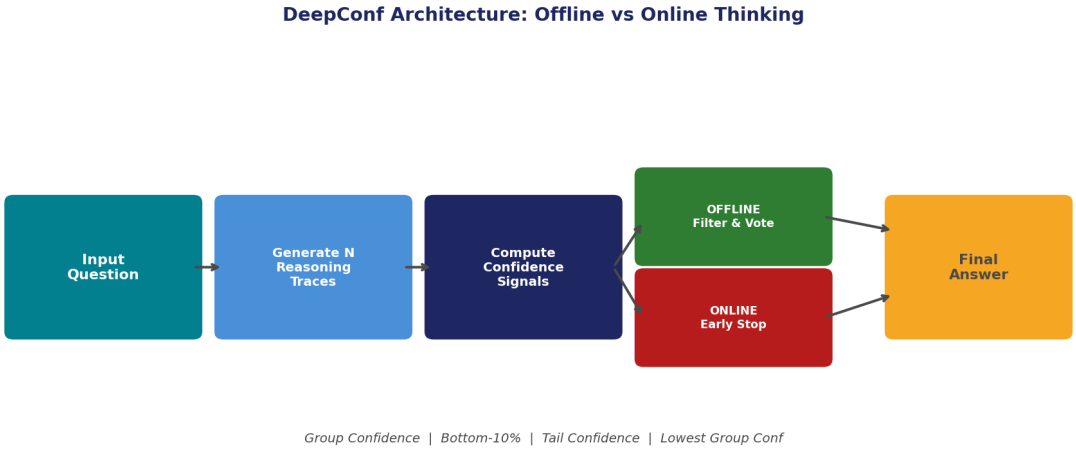

Figure 2: System overview — offline mode re-weights completed traces; online mode terminates bad traces mid-generation

5.1 Offline Mode – Voting Smarter After the Fact

In offline mode, you've generated all your N traces and haven't changed anything about the process. You're only smarter in how you aggregate them. DeepConf offers two methods:

- **Confidence Weighted Voting:** Traces still vote on the responses, but now those votes have weights attached to them depending on their level of confidence. Trace confident about its prediction of response 109 holds a lot more weight compared to a trace with little confidence in predicting it.
- **Confidence Filtering:** Remove (1-eta)% of traces from the set by confidence score before voting. Top-10% (keep top 10%) and top-90% (remove bottom 10%, which will be considered "garbage data") are available. Clearly, the former option will be much more radical but also produce significantly better results; yet, at the same time, it will run a significant risk of crashing when the model will strongly feel confident and be completely wrong.

It would help to visualize the difference between top-10% and top-90% like so: "top-10%" means "I'm absolutely certain in my 51 best out of 512 traces," while "top-90%" translates to "All I want is to get rid of my 51 most uncertain traces."

5.2 Online Mode — Stop Bad Traces Before They Finish

This is the more novel part and honestly what I found most interesting from an engineering standpoint. Instead of generating every trace all the way to the end and then filtering, you **kill traces mid-generation** when their current local confidence drops too low. Here's how it works step by step:

- **Warmup (N_init = 16 traces):** Run 16 traces to completion first. Use their lowest group confidence values to compute a stopping threshold s — specifically the percentile cutoff that corresponds to the top-eta% of warmup traces. This becomes your 'this trace is probably not worth continuing' alarm.

- **Online Generation:** For every new trace, keep computing the current group confidence window. The moment it drops below threshold s, stop the trace immediately. Don't finish generating it. Those partial tokens count toward your budget, but you save everything that would have come after.
- **Adaptive Consensus Stopping:** Also track whether the surviving traces have converged on an answer. If the top answer's vote share beta = V(best answer) / V(total) crosses 0.95, just stop generating entirely. No point continuing when you already have consensus.

5.2 Online Mode - Eliminate Unwanted Traces Early

This is the more revolutionary part of the technique, which was also my favorite part of the algorithm from an engineering point of view. Rather than generating each trace until completion before discarding, traces with low local confidence values are **killed off early on**. The way this happens is as follows:

- **Warm-Up (N_init = 16 traces):** Generate 16 traces until completion. Take the lowest group confidence values of these 16 traces and use it to compute the threshold for termination, s. This threshold is essentially the percentile of all generated traces within the top-eta% percentile range during the warm-up period. You have successfully installed your 'kill switch' to terminate traces.
- **Trace Generation:** During the generation of a new trace, continuously monitor the current group confidence window value. As soon as it falls below the threshold, s, kill the trace. Do not finish the rest of the trace; however, count these tokens against your budget.
- **Voting-based Trace Termination:** In addition to the above, also monitor whether the traces are moving towards consensus. If the vote ratio of the best response candidate, beta = V(best answer)/V(total), exceeds 0.95, stop generating traces.

The two variants — **DeepConf-low** (eta = 10%, aggressive) and **DeepConf-high** (eta = 90%, conservative) — correspond to how strict the stopping threshold is. Low stops a lot of traces early. High only stops the clearly hopeless ones.

6. Experiments and Results

The evaluation configuration looks good to me. Five tests include AIME 2024, AIME 2025 (hard mathematics contest questions), BRUMO25 (Math Olympiad at Brown University), HMMT25 (Harvard-MIT Mathematics Competition), and GPQA-Diamond (science reasoning question at graduate level). The five models involved range from tiny to massive including DeepSeek-R1-8B, Qwen3-8B, Qwen3-32B, GPT-OSS-20B, and GPT-OSS-120B. All tests are done using 64 runs per condition –

6.1 Accuracy — Offline Mode

Model / Dataset	pass@1	cons@512	DeepConf-high	DeepConf-low
GPT-OSS-120B / AIME25	91.8%	97.0%	97.0%	**99.9%**
GPT-OSS-120B / AIME24	91.9%	96.7%	96.7%	**97.0%**

DeepSeek-R1-8B / AIME25	76.9%	82.3%	81.4%	**86.4%**
DeepSeek-R1-8B / AIME24	83.0%	86.7%	86.7%	**92.5%**
Qwen3-32B / AIME25	71.7%	80.1%	80.2%	**80.2%**

However, the most impressive result was shown by GPT-OSS-120B reaching a score of 99.9% on AIME 2025, due to using DeepConf@512 with tail confidence and top-10% criteria. In comparison, a regular majority vote resulted in a mere 97%, while pass@1 scored 91.8%. Almost the best possible performance of this particular test, if you ask me.

But what caught my eye is the fact that DeepSeek-R1-8B showed a massive increase in accuracy from 82.3%, when using the regular voting method with 512 traces, to 87.4%, when applying DeepConf – that's more than five percentage points! – despite having only eight billion parameters in total.

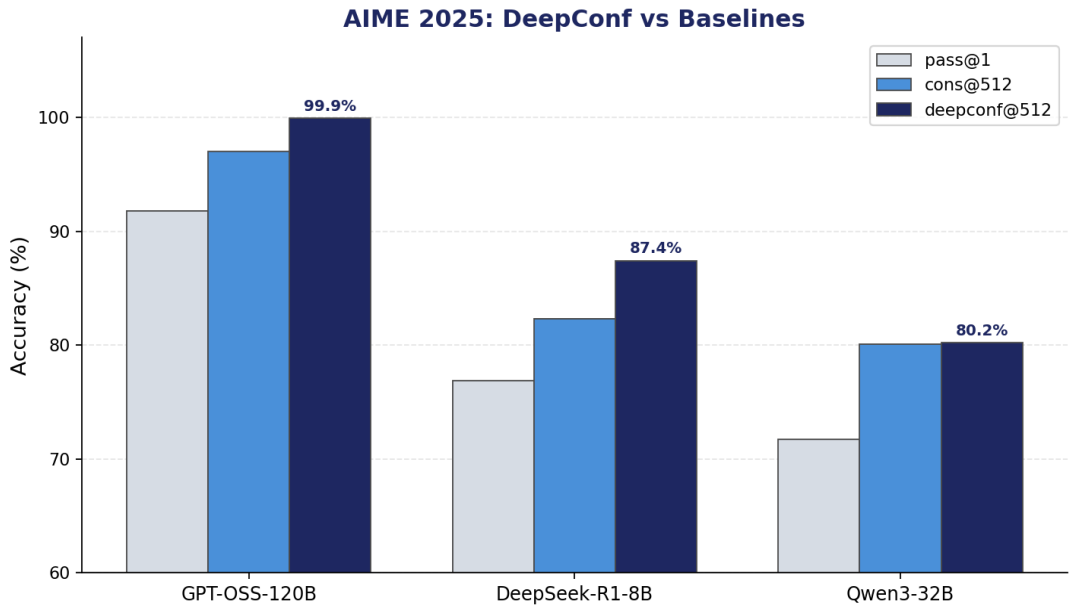

Figure 3: Accuracy comparison across models on AIME 2025 — DeepConf consistently leads over both pass@1 and cons@512

6.2 Token Efficiency — Online Mode

Task	cons@512 (x10^8)	DC-high (x10^8)	DC-low (x10^8)	Token Saving
AIME24	2.66	1.20 (-54.6%)	0.53	**-79.9%**
AIME25	3.23	1.42 (-56.0%)	0.49	**-84.7%**
BRUMO25	2.68	1.81 (-32.6%)	0.73	**-72.8%**
HMMT25	4.09	2.78 (-32.0%)	0.97	**-76.2%**

However, what impressed me the most was the extent of token reductions achieved by the models. For example, on the GPT-OSS-120B-AIME 2025 test set, DeepConf-low uses 84.7% fewer tokens compared to regular majority voting but achieves superior accuracy (97.9% vs. 97.1%). This is not a compromise; this is a straightforward upgrade across both dimensions. Moreover, even the conservative version of DeepConf (i.e., DeepConf-high) eliminates 56% of the tokens while maintaining accuracy parity.

On average, for each task in DeepSeek-R1-8B, DeepConf-low reduces token usage by 64-78%, whereas DeepConf-high reduces usage by 23-59%. Token reductions are highest in the case of the most difficult tasks, namely AIME25 and AIME24, which is logical since it is precisely in such tasks that the greatest number of traces will have to be abandoned due to their confusion.

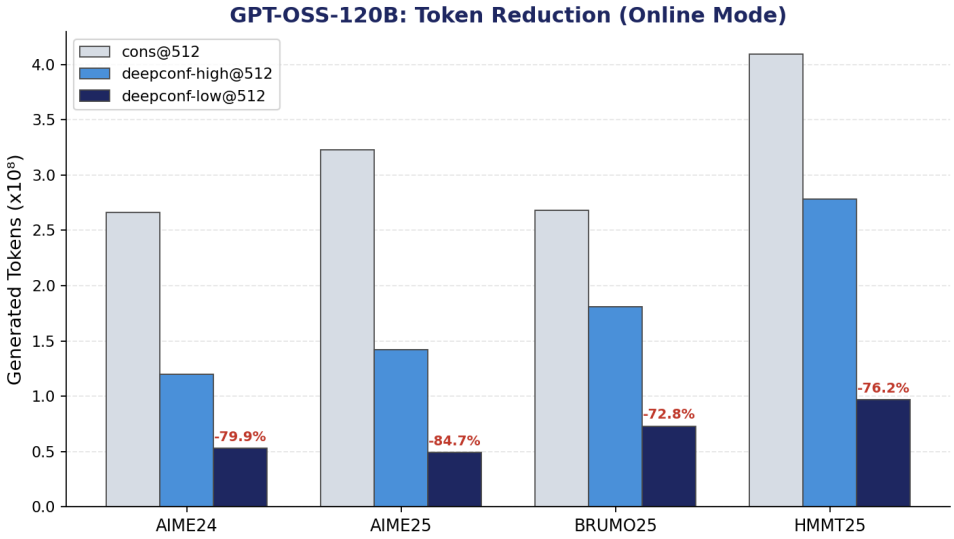

Figure 4: Generated tokens (x10^8) across tasks for GPT-OSS-120B — DeepConf-low cuts up to 84.7% of compute with no accuracy loss

6.3 Scaling Behavior

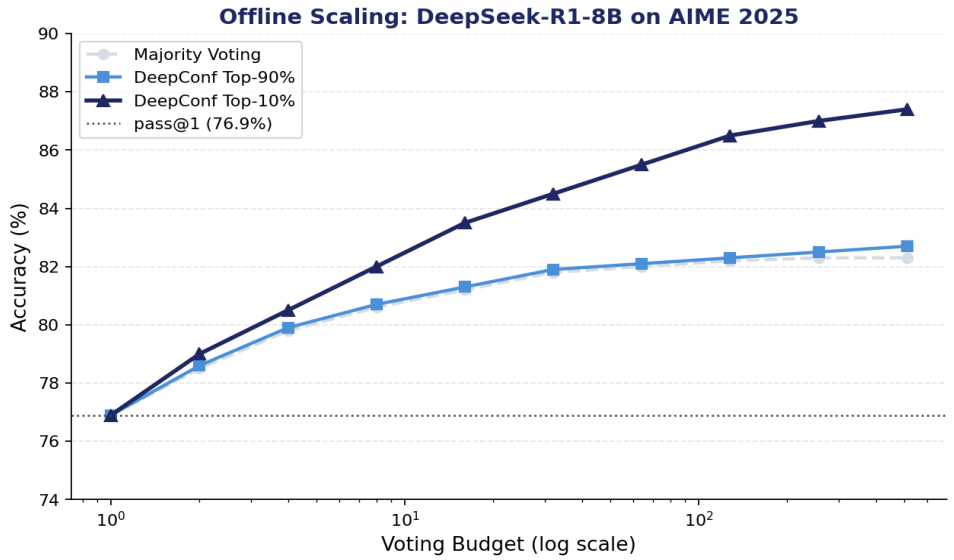

Figure 5: Offline accuracy vs. voting budget (log scale) for DeepSeek-R1-8B on AIME 2025 — DeepConf-top-10% pulls ahead across all budget levels

The scaling plots look promising. Majority vote with a very small budget (8-16 traces) is inferior to the confidence-filtering approach. The gap between the two approaches becomes more pronounced with the increase in the budget size. In particular, the effect is much stronger with top-10% filtering.

7. My Analysis and Takeaways

7.1 What I Actually Found Clever Here

The key takeaway from the paper here is how practically "free" this technique seems to be. The probability logarithms used for filtering in DeepConf are calculated anyway as part of the standard procedure for generation. The computation costs nothing extra and involves no post-processing, no external models, no additional APIs, and no verifier calls. It's all built into the process as is.

Finally, I want to return to the point that local rather than global measurements seem to give better results. This is because when solving problems myself, I experience a feeling of uncertainty after taking some steps, long after my doubt may have been erased in favor of a final solution, but this uncertainty still influences my decision making. This kind of behavior should be replicated by a model; hence, the low confidence estimates during the token selection. However, in case of global averaging, such moments would be averaged out with high confidence estimates.

7.2 The Limitations I Think Are Worth Flagging

- **The overconfidence problem is real.** An ablation from the paper itself provides examples where top-10% filtering fails – GPT-OSS-120B on HMMT25 is one of such examples. In this case, a confidently wrong model results in votes for incorrect solutions being concentrated. Correlation between confidence and correctness does exist, but they do not coincide. This method lacks any means of distinguishing the two.
- **The warmup cost bites at small budgets.** The number of traces used for warmup is fixed and equal to N_init = 16 for any budget. For a budget B = 32, you lose half of it before seeing any gain in efficiency. The authors are honest about this; however, N_init = 16 seems like a fair choice of a default parameter. Still, it puts some ceiling on possible savings on lower compute budgets..
- **Benchmarks consist only of mathematical problems.** Whether we talk about AIME, HMMT, BRUMO, or GPQA, we deal with problems that have definite and clear solutions. I am honestly interested to see how filtering would work in case of tasks like code generation, long reasoning, or summarization when there is no single true solution.
- **Tuning eta can be a pain in the rear end.** The value for eta (retention ratio; 10% vs. 90%) depends on the job at hand and the model being used. From their ablation study, there seems to be an optimal value, but this will change. In any real-world application, you will have to tune it by having a validation dataset with labels, or simply play it safe.

7.3 How This Connects to What We Covered in Class

There were a few links I noticed while going through this paper:

- **Uncertainty Quantification:** Token Entropy/Confidence is effectively estimating the epistemic uncertainty of neural networks. We discussed it in the context of Bayesian Neural Networks and Monte Carlo Dropout; DeepConf takes the exact same idea but reads it straight off the softmax probabilities of a pre-existing neural network.

- **Ensemble Methods:** The majority voting procedure over N independent traces is equivalent to an ensemble of N forward passes. DeepConf implements a weighted ensemble with pruning, which ties back directly to all the discussion on Boosting techniques and weighted/voting aggregations.

- **Trade-off between Inference vs Training Compute for Accuracy:** An underlying theme across several papers in recent large language model research is the use of Inference Compute as an effective alternative for improving accuracy over Training Compute. DeepConf makes the most of this trade-off.

- **Adaptive Computation:** Consensus stopping criteria, where token generation stops if agreement > 0.95, is a form of Adaptive Computation Depth, where more computation time is spent for difficult examples and less time on easy samples. This is related to Early Exit Networks/Early Stopping and Adaptive Transformers.

7.4 Where I Think This Goes Next

The authors mention extending this approach to RL settings and applying confidence-based early stopping for pruning policy rollouts. I think it's intriguing, yet what caught my eye was how it ties into speculative decoding. Currently, speculative decoding leverages a lightweight draft model to make token predictions, which get verified by a larger model. However, what if you utilized group confidence drops as another indicator to know when to stop relying on the predictions from the draft model?

Moreover, instead of simply pruning traces based on their confidence drops, is there a way to take action in response to confidence drops? For instance, suppose you have a drop in group confidence at a certain point in time during a trace; you could prompt the agent to correct itself or remind it that it needs to verify its recent steps.

8. Related Work

8.1 Self-Consistency and Parallel Thinking

This builds on the groundbreaking work done by Wang et al. (2023) in the self-consistency paper, which proves that voting amongst multiple reasoning chains increases accuracy. Brown et al. (2024) came next by studying how Best-of-N scaling worked, which is highlighted in the "Large Language Monkeys" paper. They found that scaling inference samples was an effective replacement for increasing the model size. This problem of efficiency has been addressed through consensus-stopping by Aggarwal et al. (2023), Xue et al. (2023), and Fu et al. (2024).

8.2 Confidence Estimation

Geng et al. (2024) offers the survey-level perspective on confidence calibration in LLMs. The work of Fadeeva et al. (2024), who used token-level uncertainty in fact-checking, is the closest one to ours. The closest work to ours in terms of relevance is Self-Certainty by Kang et al. (2025). DeepConf’s contribution compared to that of Self-Certainty is the use of local measurements and online early stopping – in contrast, Self-Certainty requires the whole trace to evaluate its score.

8.3 Efficient Reasoning Without Retraining

ThinkPrune (Hou et al., 2025) performs pruning for long chain-of-thought through RL fine-tuning. O1-Pruner (Luo et al., 2025) is engaged in length-harmonizing fine-tuning. The empirical study conducted by Chen et al. (2024) reveals that calling on LLMs again and again does not improve the results; there come point of diminishing returns pretty quickly. The unique feature of DeepConf among all of those: training is unnecessary.

9. Conclusion

In all honesty, DeepConf is a paper whose premise is easy enough to state in two sentences but whose execution is meticulous and whose results are solid. First, the insight, which is intuitive and backed up by empirical evidence, is that the model's internal token probability distributions are enough to identify its reliable reasoning traces. Second, the practicality comes from being able to use this signal online to filter out unreliable traces while they're being generated, instead of as an offline analysis tool after the fact.

The numbers say it all. 84.7% reduction in token usage with no accuracy loss for the largest model, 5+ accuracy gain for 8B models with no additional computational cost for inference. These results speak for themselves for a method that doesn't require any modifications to the model itself nor any further training.

For the future, my main concern would be the overconfidence problem, which arises from the method's inability to differentiate between "confident and correct" and "confident and wrong". This assumption may break down when confidence calibration methods are introduced or if DeepConf is used in conjunction with a light correctness indicator. Nevertheless, the method's performance is impressive, and I expect it to be adopted quickly.

References

[1] Wang et al. (2023). Self-Consistency Improves Chain of Thought Reasoning in Language Models. arXiv:2203.11171.

[2] Kang, Zhao & Song (2025). Scalable Best-of-N Selection for Large Language Models via Self-Certainty. arXiv:2502.18581.

[3] Brown et al. (2024). Large Language Monkeys: Scaling Inference Compute with Repeated Sampling. arXiv:2407.21787.

[4] Guo et al. (2025). DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. arXiv:2501.12948.

[5] Yang et al. (2025). Qwen3 Technical Report. arXiv:2505.09388.

[6] Fadeeva et al. (2024). Fact-Checking the Output of LLMs via Token-Level Uncertainty Quantification. arXiv:2403.04696.

[7] Chen et al. (2024). Are More LLM Calls All You Need? Towards Scaling Laws of Compound Inference Systems. arXiv:2403.02419.

[8] Fu et al. (2024). Efficiently Scaling LLM Reasoning with CertaIndex. arXiv:2412.20993.

[9] Snell et al. (2024). Scaling LLM Test-Time Compute Optimally Can Be More Effective Than Scaling Model Parameters. arXiv:2408.03314.

[10] Hou et al. (2025). ThinkPrune: Pruning Long Chain-of-Thought of LLMs via Reinforcement Learning. arXiv:2504.01296.